

SECURE CHANNELS

Technical Documentation

Chat Application — Built for Bidyut Innovations

Version	0.1.0
Framework	Next.js 16 • React 19
Real-Time	Pusher Channels
Auth	Firebase Authentication
Database	MongoDB / Mongoose
Status	Active Development

1. Application Overview

Secure Channels is a full-stack real-time chat platform developed for Bidyut Innovations. It enables users to create or join private chat rooms using unique 6-character alphanumeric room codes. The application is built on Next.js 16 with the App Router, uses Firebase for authentication, Pusher for real-time event delivery, and MongoDB for message persistence.

Core Features

Feature	Description
Firebase Authentication	Email/password sign-up and login, session persisted across page loads.
Room-Based Chat	Generate a random room code or join an existing room by entering the code.
Real-Time Messaging	Pusher broadcasts messages instantly to all subscribers in a room.
Message Persistence	MongoDB stores messages; last 50 are loaded on room join.
Optimistic UI	Messages appear in the sender's UI immediately before server confirmation.
Session Persistence	Room ID saved in sessionStorage so page refreshes keep the user in their room.
Responsive Dark UI	Premium glassmorphism design built with Tailwind CSS v4 and shadcn/ui.
Room Code Sharing	One-click clipboard copy of the room code for easy sharing.

2. Technology Stack

Layer	Technology	Version / Notes
Framework	Next.js (App Router)	16.2.0
UI Library	React	19.2.4
Styling	Tailwind CSS + shadcn/ui	v4 + tw-animate-css
Authentication	Firebase Auth	12.11.0
Real-Time	Pusher Channels	Server 5.3.3 / Client 8.4.3
Database	MongoDB / Mongoose	9.3.1
Icons	Lucide React	0.577.0
Language	JavaScript + TypeScript	TypeScript 5
Server	Node.js custom server	server.js (Socket.IO reference)

3. Project Structure

```
chat-app/
├── app/
│   ├── api/
│   │   └── messages/route.js    # REST API: GET & POST messages
│   ├── globals.css             # Global Tailwind styles
│   ├── layout.js               # Root HTML layout
│   └── page.js                 # Main page: auth / home / chat screens
├── src/
│   ├── components/
│   │   ├── MessageBubble.jsx   # Chat message component
│   │   └── ui/                 # shadcn/ui primitives
│   ├── hooks/
│   │   └── useSocket.js         # Pusher hook + sendMessage logic
│   ├── lib/
│   │   ├── firebase.js         # Firebase app + Auth init
│   │   ├── mongodb.js         # MongoDB connection helper
│   │   ├── pusher.js           # Pusher server + client instances
│   │   └── utils.js            # Shared utilities
│   └── models/
│       └── Message.js          # Mongoose schema
├── server.js                   # Custom Node.js + Socket.IO server
├── package.json
├── next.config.ts
└── tsconfig.json
```

4. System Architecture

Message Flow

The following steps describe how a message travels from the sender through the system and reaches all participants in the same room.

Step	Stage	Description
1	User Action	User types a message and presses Enter or the Send button.
2	Optimistic Update	Message is appended to local React state for instant feedback.
3	HTTP POST	A POST request is sent to /api/messages with { room, author, content, time, msgId }.
4	MongoDB Write	The API route saves the message document to the MongoDB messages collection.
5	Pusher Trigger	The API calls pusherServer.trigger('chat-{room}', 'new-message', data).
6	Broadcast	Pusher broadcasts the event to all clients subscribed to the channel.
7	Deduplication	useSocket hook filters incoming events using msgId to prevent double rendering.

Step	Stage	Description
8	Render	New MessageBubble renders in every connected participant's chat view.

5. Environment Variables

Create a `.env.local` file in the project root. Variables prefixed with `NEXT_PUBLIC_` are exposed to the browser bundle.

Firebase (Client-side)

Variable	Description
<code>NEXT_PUBLIC_FIREBASE_API_KEY</code>	Firebase project API key
<code>NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN</code>	Firebase auth domain (e.g. <code>project.firebaseio.com</code>)
<code>NEXT_PUBLIC_FIREBASE_PROJECT_ID</code>	Firebase project ID
<code>NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET</code>	Firebase storage bucket
<code>NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID</code>	Firebase messaging sender ID
<code>NEXT_PUBLIC_FIREBASE_APP_ID</code>	Firebase app ID

MongoDB

Variable	Description
<code>MONGODB_URI</code>	Full MongoDB connection string (Atlas or self-hosted)

Pusher

Variable	Description
<code>PUSHER_APP_ID</code>	Server-side Pusher App ID
<code>PUSHER_SECRET</code>	Server-side Pusher secret key
<code>NEXT_PUBLIC_PUSHER_KEY</code>	Client-side Pusher public key
<code>NEXT_PUBLIC_PUSHER_CLUSTER</code>	Pusher cluster region (e.g. <code>ap2</code>)

6. API Reference

GET `/api/messages`

Returns the last 50 messages for a given room, sorted oldest to newest.

Parameter	Type	Required	Description
room	Query string	Yes	The room ID (e.g. ABC123)

Success Response (200)

```
{
  "success": true,
  "messages": [
    {
      "_id": "665f...",
      "room": "ABC123",
      "author": "john",
      "content": "Hello!",
      "time": "10:32 AM",
      "msgId": "abc123def456",
      "createdAt": "2024-06-01T10:32:00.000Z"
    }
  ]
}
```

POST /api/messages

Saves a new message to MongoDB, then broadcasts it via Pusher to all room subscribers.

Field	Type	Required	Description
room	String	Yes	Room ID
author	String	Yes	Username (derived from Firebase email prefix)
content	String	Yes	Message text
time	String	Yes	Formatted display time (e.g. 10:32 AM)
msgId	String	No	Client-generated unique ID for deduplication

7. Key Components

page.js — Main Page Controller

Manages three UI screens using a React screen state variable:

Screen	Description
auth	Firebase email/password sign-in and new account registration.
home	Dashboard to create a new room or join one by entering a room code.
chat	Active chat room with real-time message list and message input bar.

useSocket.js — Real-Time Hook

- Subscribes to the Pusher channel chat-{room} when a room is joined.
- Fetches message history via GET /api/messages on mount.
- Exposes sendMessage(author, content) which applies an optimistic UI update then POSTs to the API.
- Deduplicates incoming Pusher events using the msgId field to prevent double rendering.

MessageBubble.jsx — Message Renderer

Stateless component rendering a single chat message. Own messages appear right-aligned in blue; others appear left-aligned in slate grey.

route.js — API Route Handler

Next.js App Router handler at /api/messages. GET returns paginated history; POST saves to MongoDB and triggers a Pusher broadcast.

Message.js — Mongoose Schema

Defines the MongoDB document shape with fields: room, author, content, time, msgId (indexed), and auto-managed createdAt / updatedAt timestamps.

firebase.js — Firebase Init

Initialises the Firebase singleton and exports the Auth instance used for all sign-in and sign-out operations.

8. Data Model

Message Schema (MongoDB / Mongoose)

Field	Type	Required	Indexed	Description
room	String	Yes	No	Room ID the message belongs to
author	String	Yes	No	Display name (email prefix)
content	String	Yes	No	Message body text
time	String	Yes	No	Human-readable time string
msgId	String	No	Yes	Client UUID for deduplication
createdAt	Date	Auto	No	Managed by Mongoose timestamps
updatedAt	Date	Auto	No	Managed by Mongoose timestamps

9. Getting Started

Prerequisites

- Node.js 18 or later
- npm (bundled with Node.js)
- A MongoDB Atlas cluster (or local MongoDB instance)
- A Firebase project with Email/Password authentication enabled
- A Pusher account with a Channels app created

Installation

```
# 1. Clone the repository
git clone <your-repo-url>
cd chat-app

# 2. Install dependencies
npm install

# 3. Create environment file
cp .env.example .env.local
# Fill in your Firebase, MongoDB, and Pusher credentials

# 4. Start development server
npm run dev

# App is now running at http://localhost:3000
```

Available Scripts

Command	Description
npm run dev	Start Next.js in development mode with hot reload
npm run build	Compile the application for production
npm run start	Start the production Next.js server
npm run lint	Run ESLint across the codebase
node server.js	Start the custom Node.js + Socket.IO server (reference)

10. Deployment

Vercel (Recommended)

Vercel is the preferred deployment platform for Next.js. API routes are automatically deployed as serverless functions.

1. Push the repository to GitHub (or GitLab / Bitbucket).
2. Import the project at vercel.com/new.
3. Add all environment variables under Project → Settings → Environment Variables.

4. Click Deploy. Vercel handles the build and CDN distribution automatically.

Self-Hosted (Node.js + PM2)

```
# Build the application
npm run build

# Install PM2 process manager globally
npm install -g pm2

# Start the custom server in production mode
NODE_ENV=production pm2 start server.js --name chat-app

# Persist process list and enable auto-restart on reboot
pm2 save
pm2 startup
```

This document is confidential and intended solely for internal use by Bidyut Innovations. All rights reserved.